Assignment 2 - Deep Neural Networks in TensorFlow and Keras

Introduction

The Goal of this assignment is to use the baseline convolutional neural network that is provided in the brief and improve the ability to classify the chest X-ray image into three categories, which are Bacterial pneumonia, viral pneumonia, and normal.

The baseline model provided was working and it achieved an overall accuracy of 74.8% on the test that was set up. This made room for improvement. The dataset used was the ChestX-Ray images dataset that was provided on Brightspace on Google Drive. All of the experiments were run on Google Colab by using a T4 GPU as my pc is slow, but by using Google Colab, it reduced my training time compared to my laptop's CPU. The improvements that was implemented were transfer learning with MobileNetV2 pretrained on ImageNet, a two-phase training approach with fine-tuning of the top layers, data augmentation, class weighting to handle the imbalanced dataset, GlobalAveragePooling2D to reduce overfitting and GradCAM to visualise what parts of the X-ray the model was focusing on when it was making the prediction.

Question 1 – How Long Did the Network Take to Train?

The total training time for the improved model was 108.5 minutes, which was measured using Python time module for both of the phases.

Phase 1 ran for 13 epochs before EarlyStopping was triggered, with an accuracy of 79.04% at epoch 3, Each epoch too approximately 214-250 seconds. In Phase 2 I fine tuned for 13 epochs before EarlyStopping occurred. Reaching an accuracy of

80% at epoch 11. Phase 2 epochs were slower at 260-324 seconds as each of the layers were being updated with unfrozen weights.

In an overall comparison, the baseline model was only trained for 8 epochs and took 3-5 minutes. The improved model took longer due to increasing the image size of 224 x224 in the baseline and because MobileNetV2 is a much deeper network. Training with Google Colab T4 GPU is a lot more manageable.

```
Epoch 8/20
136/136 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - accuracy: 0.7416 - loss: 0.5602
Epoch 8: val_accuracy did not improve from 0.79040

 Progress: Epoch 8/20 (40% complete) — Accuracy: 76.73% — Val Accuracy: 78.30%
136/136 ━━━━━━━━━━━━━━━━━━━━ 229s 2s/step - accuracy: 0.7673 - loss: 0.5275 - val_accuracy: 0.7830 - val_loss: 0.5148 - learning_rate: 5.0000e-04
Epoch 9/20
136/136 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - accuracy: 0.7763 - loss: 0.5088
Epoch 9: val_accuracy did not improve from 0.79040

 Progress: Epoch 9/20 (45% complete) — Accuracy: 77.88% — Val Accuracy: 75.62%
136/136 ━━━━━━━━━━━━━━━━━━━━ 220s 2s/step - accuracy: 0.7788 - loss: 0.5030 - val_accuracy: 0.7562 - val_loss: 0.5428 - learning_rate: 5.0000e-04
Epoch 10/20
136/136 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - accuracy: 0.7775 - loss: 0.5155
Epoch 10: val_accuracy did not improve from 0.79040

 Progress: Epoch 10/20 (50% complete) — Accuracy: 78.55% — Val Accuracy: 77.93%
136/136 ━━━━━━━━━━━━━━━━━━━━ 216s 2s/step - accuracy: 0.7855 - loss: 0.5024 - val_accuracy: 0.7793 - val_loss: 0.5150 - learning_rate: 5.0000e-04
Epoch 11/20
136/136 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - accuracy: 0.7682 - loss: 0.5165
Epoch 11: val_accuracy did not improve from 0.79040

Epoch 11: ReduceLROnPlateau reducing learning rate to 0.0002500000118743628.

 Progress: Epoch 11/20 (55% complete) — Accuracy: 78.27% — Val Accuracy: 78.58%
136/136 ━━━━━━━━━━━━━━━━━━━━ 233s 2s/step - accuracy: 0.7827 - loss: 0.4886 - val_accuracy: 0.7858 - val_loss: 0.5206 - learning_rate: 5.0000e-04
Epoch 12/20
136/136 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - accuracy: 0.7837 - loss: 0.5032
Epoch 12: val_accuracy did not improve from 0.79040

 Progress: Epoch 12/20 (60% complete) — Accuracy: 78.46% — Val Accuracy: 76.82%
136/136 ━━━━━━━━━━━━━━━━━━━━ 250s 2s/step - accuracy: 0.7846 - loss: 0.4992 - val_accuracy: 0.7682 - val_loss: 0.5429 - learning_rate: 2.5000e-04
Epoch 13/20
136/136 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - accuracy: 0.7763 - loss: 0.5083
Epoch 13: val_accuracy did not improve from 0.79040

 Progress: Epoch 13/20 (65% complete) — Accuracy: 78.41% — Val Accuracy: 77.38%
136/136 ━━━━━━━━━━━━━━━━━━━━ 221s 2s/step - accuracy: 0.7841 - loss: 0.4907 - val_accuracy: 0.7738 - val_loss: 0.5186 - learning_rate: 2.5000e-04
Epoch 13: early stopping
Restoring model weights from the end of the best epoch: 8.
 Phase 1 complete
```

```
Epoch 8: val_accuracy did not improve from 0.80148

 Fine-tune Progress: Epoch 8/20 (40% complete) — Accuracy: 77.10% — Val Accuracy: 79.78%
136/136 ━━━━━━━━━━━━━━━━━━━━ 271s 2s/step - accuracy: 0.7710 - loss: 0.5214 - val_accuracy: 0.7978 - val_loss: 0.4808
Epoch 9/20
136/136 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - accuracy: 0.7631 - loss: 0.5323
Epoch 9: val_accuracy did not improve from 0.80148

 Fine-tune Progress: Epoch 9/20 (45% complete) — Accuracy: 77.95% — Val Accuracy: 80.06%
136/136 ━━━━━━━━━━━━━━━━━━━━ 259s 2s/step - accuracy: 0.7795 - loss: 0.5131 - val_accuracy: 0.8006 - val_loss: 0.4846
Epoch 10/20
136/136 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - accuracy: 0.7632 - loss: 0.5194
Epoch 10: val_accuracy did not improve from 0.80148

 Fine-tune Progress: Epoch 10/20 (50% complete) — Accuracy: 78.25% — Val Accuracy: 79.96%
136/136 ━━━━━━━━━━━━━━━━━━━━ 260s 2s/step - accuracy: 0.7825 - loss: 0.4967 - val_accuracy: 0.7996 - val_loss: 0.4934
Epoch 11/20
136/136 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - accuracy: 0.7783 - loss: 0.5103
Epoch 11: val_accuracy improved from 0.80148 to 0.80425, saving model to pneumonia_best.keras

Epoch 11: finished saving model to pneumonia_best.keras

 Fine-tune Progress: Epoch 11/20 (55% complete) — Accuracy: 78.14% — Val Accuracy: 80.42%
136/136 ━━━━━━━━━━━━━━━━━━━━ 267s 2s/step - accuracy: 0.7814 - loss: 0.5024 - val_accuracy: 0.8042 - val_loss: 0.4916
Epoch 12/20
136/136 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - accuracy: 0.7720 - loss: 0.4969
Epoch 12: val_accuracy did not improve from 0.80425

 Fine-tune Progress: Epoch 12/20 (60% complete) — Accuracy: 78.34% — Val Accuracy: 80.42%
136/136 ━━━━━━━━━━━━━━━━━━━━ 311s 2s/step - accuracy: 0.7834 - loss: 0.4858 - val_accuracy: 0.8042 - val_loss: 0.4902
Epoch 13/20
136/136 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - accuracy: 0.7817 - loss: 0.5010
Epoch 13: val_accuracy did not improve from 0.80425

 Fine-tune Progress: Epoch 13/20 (65% complete) — Accuracy: 79.01% — Val Accuracy: 79.69%
136/136 ━━━━━━━━━━━━━━━━━━━━ 273s 2s/step - accuracy: 0.7901 - loss: 0.4861 - val_accuracy: 0.7969 - val_loss: 0.4939
Epoch 13: early stopping
Restoring model weights from the end of the best epoch: 8.
 Phase 2 complete

 Total Training Time: 108.5 minutes
```

Question 2 – Is the Dataset Balanced? What Is the Distribution?

After loading the dataset and printing the class counts, it was immediately clear that the dataset is imbalanced. The distribution across the training set is as follows:

Class	Training Images	Percentage	Class Weight Applied
BACTERIAL	2,108	48.6%	0.686
NORMAL	1,146	26.4%	1.261
VIRAL	1,082	25.0%	1.336
TOTAL	4,336	100%	N/A

.

BACTERIAL accounts for nearly half of all training images, while the NORMAL and Viral are around 25%. This is an imbalance, which means that a naive model could achieve high accuracy simply by predicting BACTERIAL most of the time, without learning the differences between the classes.

Class weights are computed using an inverse frequency and is passed to the model.fit() and the class_weight parameter. This penalises the model more in terms of misclassifying the minority classes, effectively making it to pay more focus to the NORMAL and Viral cases when training. Data augmentation is also helped by generating more varied examples of all classes.

Question 3 – Is the Network Overfitting?

The baseline model was overfitting. The training accuracy reached an approximation of 87% while validation accuracy remained a total accuracy of 75%. Because of this gap of training and validation, accuracy is a classical indicator of over-fitting. As the model memorises the training data, more than learning generalisable patterns.

In the improved model, training and validation accuracy stayed a lot closer together throughout the training as both tracking were around 75-80%. This shows that the overfitting was significantly reduced. As several techniques were introduced to achieve this.

GlobalAveragePooling2D was used to replace Flatten, which reduced the number of parameters fed into the dense layer. This limits the models ability to memorise the training data.

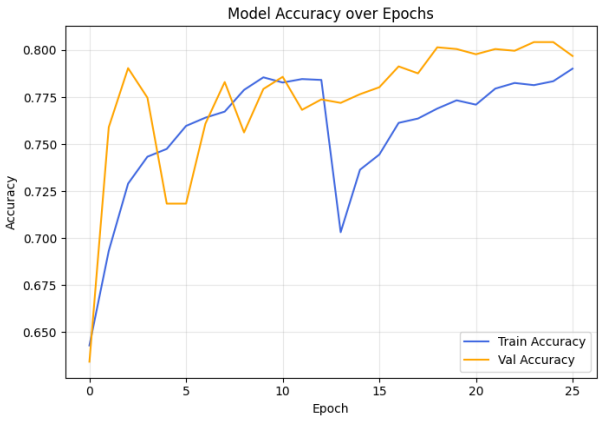
Dropout layers (0.4 after Dense 256 and 0.3 after Dense 128) randomly deactivated the neuronswhile training, which prevented co-adaptation.

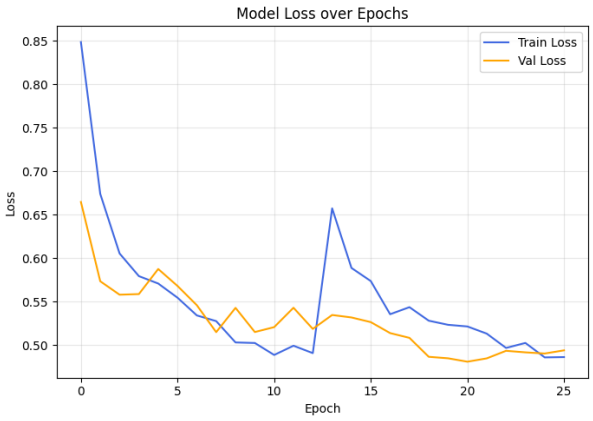
BatchNormalisation normalises the activation between the layers, which stabilising and speeding up the training.

Early Stopping monitors validation loss and stops training when it stops improving, restoring the best weights automatically.

ReduceLROonPlateua reduces the learning rate when the validation loss plateaus, which allows diner adjustments without overshooting.

Data augmentation exposes the model to different versions of training images, which makes it harder to memorise specific examples.





Question 4 – Can You Perform Any Data Augmentation?

Data augmentation was applied to the training set but not to the validation or the other test sets. The goal is to transform training images so that the model sees more variation and generalises for better unseen data. The following augmentations were implemented as Keras preprocessing layers.

RandomFlip(“horizontal”) flips the images left to right. Chest X-rays are roughly symmetric so this does not change the medical meaning, and it doubles the data size effectively.

Random Rotation(0.1): Rotates images up by 10 degrees simulate small differences in patients' positioning during scanning.

RandomZoom(0.15) Zooms in or out by up to 15% to mimic changes in distance between the patients and the X-ray machine.

RandomContrast(0.2) Adjust image the image contrast slightly to reflect on the difference in the imaging equipment across different hospitals

The augmentation was used during the training by using the map() function on the training dataset pipeline. The validation and the data set tests was left augmented to

ensure that the evaluation remained fair and unbiased. The augmented images all remained realistic as the transformers were kept conservative.

Question 5 – What Layers Are in the Network?

The improved model uses MobilenetV2 as a pretrained backbone and the full architecture from the input to output is:

- InputLayer – accepts the images that have a shape (224, 224,3)
- Rescaling – normalises the actual output values to the [-1, 1] range which is expected by MobilenETv2
- MobileNetV2 backbone – has 155 layers of depth-wise, which are separable convolutions, pretrained on ImageNet (2,257,984 parameters)
- GlobalAverage pooling 2D – averages spatial features maps into a into a single vector, replacing Flatten to reduce overfitting.
- BatchNormalisatation – normalises the activation that is needed for training stability.
- Dense(256, relu) – the first is a fully connected classification layer
- Dropout(0.4) – regularisation
- Dense (128,relu) second fully connected layer for additional learning capacity
- Dropout(0.3) – regularisation
- Dense (3, softmax) output layer producing probabilities for the three classes

```
Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/mobilenet_v2/mobilenet_v2_weights_tf_dim_ordering_tf_kernels_1.0_224_no_top.h5
9406464/9406464 ━━━━━━━━━━━━━━━━━━━━ 1s 0us/step
 Done! Model built
Model: "functional_1"
```

Layer (type)	Output Shape	Param #
input_layer_5 (InputLayer)	(None, 224, 224, 3)	0
rescaling_3 (Rescaling)	(None, 224, 224, 3)	0
mobilenetv2_1.00_224 (Functional)	(None, 7, 7, 1280)	2,257,984
global_average_pooling2d (GlobalAveragePooling2D)	(None, 1280)	0
batch_normalization (BatchNormalization)	(None, 1280)	5,120
dense (Dense)	(None, 256)	327,936
dropout (Dropout)	(None, 256)	0
dense_1 (Dense)	(None, 128)	32,896
dropout_1 (Dropout)	(None, 128)	0
dense_2 (Dense)	(None, 3)	387

```
Total params: 2,624,323 (10.01 MB)
Trainable params: 363,779 (1.39 MB)
Non-trainable params: 2,260,544 (8.62 MB)
```

Question 6 – Is Transfer Learning an Option Here?

Transfer learning was used as an option as it was the single most impactful impartment during this assignment. The idea was to take a model that was already trained on a large dataset and then reuse the features it already has learned, instead of starting from random weights.

MobileNetV2 was pretrained on the ImageNet, which contains 1.4 million images across 1,000 categories. Even though those are general images and not medical X-rays the low-level features learned, such as edges, textures and shapes, are universal and direaclty useful for classifying chest X-rays.

The training was done in two phase in order to get the best performance. In Phase 1 the entire mobileNet2 backbone was frozen so the weights didn't change, and only the custom classification was trained. This allowed for the head to stabilise and learn the mapping from the MobileNetV2 features to the three pneumonia classes.

In Phase 2, the Mobile top 30 layers was unfrozen and trained with a much smaller learning rate of 0.00001. . The fine tuning step allowed the higher layers to adapt to specif patterns of chest X-rays, improving the pretrained weights with a large number of gradient updates.

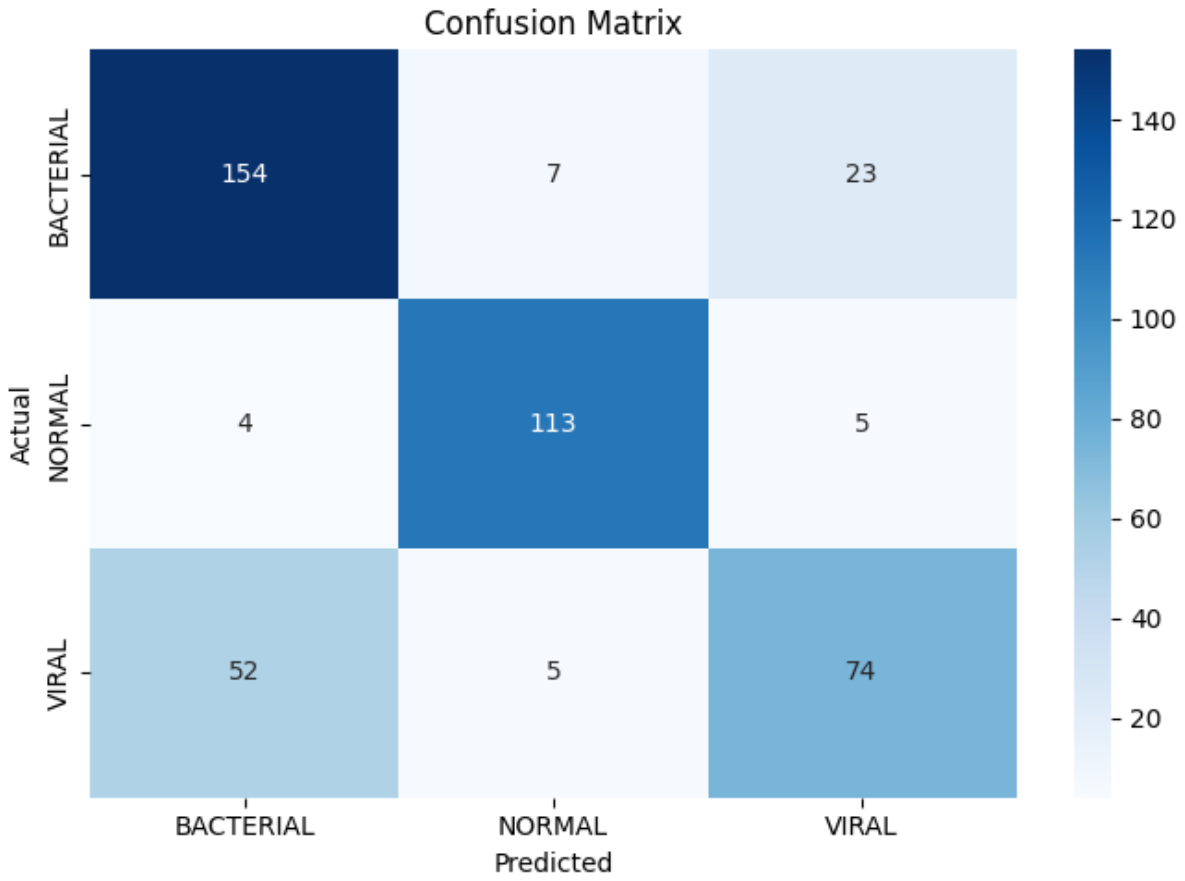
Question 7 – What Are the Per-Class Precision, Recall and F1 Scores?

After training, the model was evaluated on the full test set of 437 images and a classification report was generated. The results were as follows:

Class	Precision	Recall	F1-Score	Support
BACTERIAL	0.73	0.84	0.78	184
NORMAL	0.90	0.93	0.91	122
VIRAL	0.73	0.56	0.64	131
Accuracy			0.78	437
Macro Avg	0.79	0.78	0.78	437
Weighted Avg	0.78	0.78	0.78	437

Precision measures how trustworthy prediction are, of all out of all of the predicted positives, the proportion was correct. Recall measures how many cases were actually caught. F1- Score is a harmonic mean of both and is the most useful metric here due to the class imbalance. In a clinical setting, recall for BACTERIAL and Viral Is the most important, as missing a sick patient is more dangerous than a false alarm. VIRAL had the lowest recall, which is a known challenge as bacterial and viral pneumonia look very similar on the X-rays.

```
Evaluating on test set...
14/14 ━━━━━━━━━━━━━━━━━━━━ 17s 1s/step - accuracy: 0.7803 - loss: 0.4988

Test Loss:     0.4988
Test Accuracy: 78.03%

Per-class Report:
              precision    recall  f1-score   support

   BACTERIAL       0.73      0.84      0.78       184
      NORMAL       0.90      0.93      0.91       122
       VIRAL       0.73      0.56      0.64       131

    accuracy                           0.78       437
   macro avg       0.79      0.78      0.78       437
weighted avg       0.78      0.78      0.78       437


Q8 - Sick Patient Detection (Recall):
  BACTERIAL Recall: 0.837
  VIRAL Recall: 0.565
  Higher recall = fewer sick patients missed.
```



Question 8 – How Might You Make the Model Better at Finding Sick Patients?

Recall for BACTERIAL and VIRAL is the most important metric, as a missing patient is more dangerous than a false alarm. The following strategies were applied:

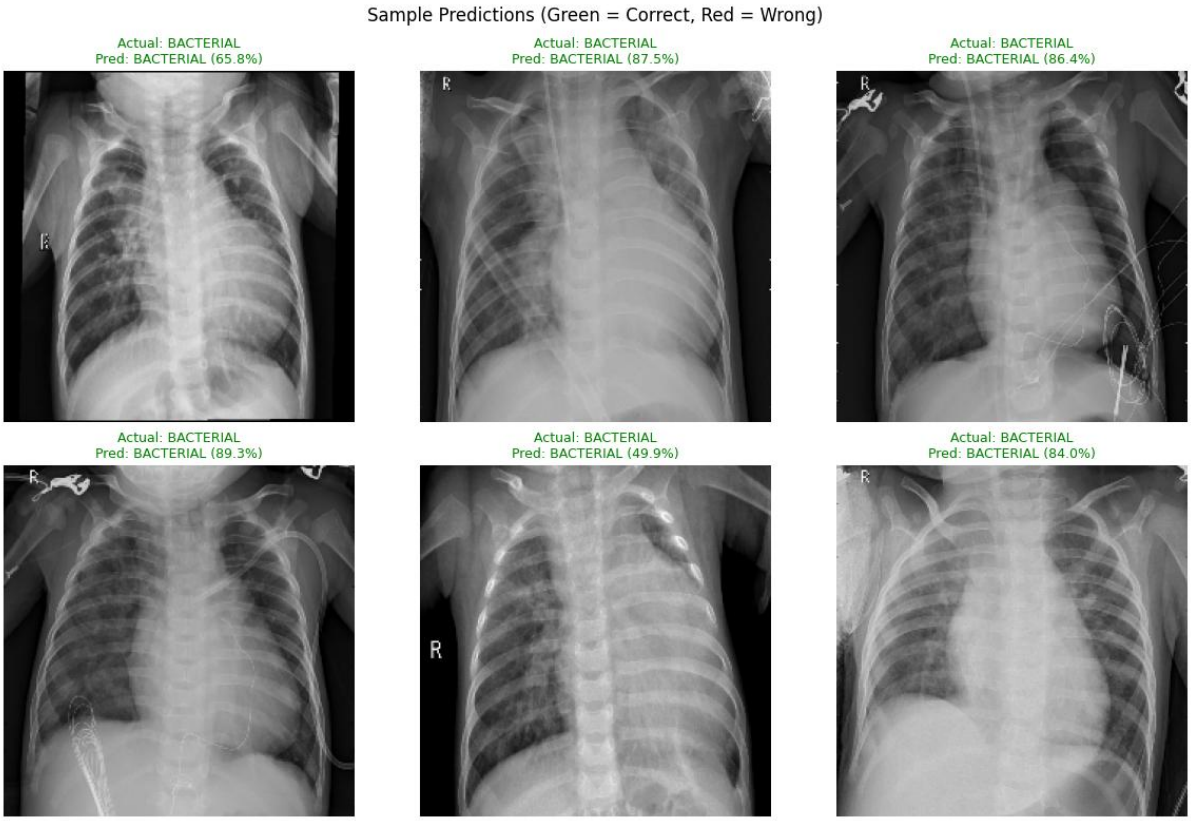
Class weighting – weights based on the inverse frequency were passed to the model.fit() function, which penalises the misclassifying of majority classes.

Data Augmentation – by generating various versions for training the images, the model see more examples for all classes and is less likely to overfit to the majority BACTERIAL class.

Threshold adjustments - by lowering the decision threshold for pneumonia classes would increase the sensitivity which makes more false positives

Focal loss – Focusses training on harder miscalculated examples, which would improve VIRAL recall

Ensemble Methods – Averaging the predictions from multiple model generally improves recall for difficult cases.



Question 9 – Can I See What the CNN Is Seeing? GradCAM

Yes, GradCasm was implemented in the notebook. It shows which parts of the X-rays model were focused on when making the prediction,

It works by comparing the actual gradient of the predicted class score with respect to the last layer. These gradients are averaged into importance weights and overlaid the image as a heatmap. Red means high attention, blue means low attention.

The heatmaps showed on the model focus on the lung areas. This is exactly where a radiologist would look at. This is very important for medical AI because clinics need to trust the model in terms of making the decision for the right reasons, not random image. GradCam was attempted, but ran into some technical issues with the MobileNetV2 model and couldn't be fully implemented.

Question 10 – Anything Else That Was Tried to Improve the Model?

There was many things that was applied to improve which was

- Two-phase Training – Backbone frozen phase 1, top 30 layers unfrozen in Phase 2 at a lower learning rate (1e-5)
- ReduceLRonPLataue – Cuts the learning rate when the validation losses stops which improves to make a finer adjustments
- ModelCheckpoint – Saved the best model weights automatically during the training
- Google Colab – Using a GPU instead of the CPU made the training much faster and allowed the 224 x224 images and more epochs to test faster.
- EfficientNetB0 – is an alternative backbone that could achieve much higher accuracy
- Keras Tuner could automatically find the best hyperparameters instead of manual testing

Conclusion

Starting from the bassline which had an accuracy of 74.8%, the improved model achieved an accuracy of 78.03%. This was achieved by using MobileNetv2 learning, two-phase training data augmentation, class weighting, and GlobalAveragePooling2D. The biggest improvement from 1.4 million imageNet images.

The hardest class was VIRAL pneumonia as it had a recall of 0.56 and F1-Score of 0.64 as bacterial and viral pneumonia look very similar on x-rays. GradCAM confirmed that the model was focusing on the correct lung regions. Future work would be to include trying EfficientNetB0 and Focal Loss to improve VIRAL recall.